Document number:	P2037r0
Date:	2020-01-11
Audience:	LEWG
Reply-to:	*Andrzej Krzemieński <akrzemi1 at gmail dot com>*

String's gratuitous assignment

This paper explores the capability of the assignment from `char` to `std::string` and the consequences of removing it.

Background

The interface of `std::basic_string` provides the following signature:

```
constexpr basic_string& operator=(charT c);
```

This allows the direct assignment from `char` to `std::string`:

```
std::string s;
s = 'A';
assert(s == "A");
```

However, due to the implicit conversion between scalar types, this allows an assignment from numeric types, such as `int` or `double`, which often has an undesired semantics:

```
std::string s;
s = 50;
assert(s == "2");

s = 48.0;
assert(s == "0");
```

In order to prevent the likely inadvertent conversions, [RU013] proposes to change the signature so that it is equivalent to:

```
template <class T>
  requires is_same_v<T, charT>
constexpr basic_string& operator=(charT c);
```

Discussion

Intended usage

Even the intended usage of the assignment from `char` is suspicious. We have a direct interface for assigning a single character to an existing `std::string`:

```
std::string s;
s = 'A';
```

However, there is no corresponding interface — in the form of constructor — for initializing a string from a single character. We have to use a more verbose syntax:

```
const std::string s1 (1u, 'C');
const std::string s2 = {'C'};
```

Whatever the motivation for the assignment from `char` was, surely the same motivation applied for the converting constructor.

Common pitfall

There are two common situations where the gratuitous converting assignment from `int` to `std::string` is used inadvertantly and results in a well-formed C++ program that does something else than what the programmer intended.

First is when inexperienced programmers try to use their experience from weakly typed languages when trying to convert from `int` to `std::string` through an assignment syntax:

```
template <typename From, typename To>
  requires std::is_assignable_v<To&, From const&>
void convert(From const& from, To& to)
{
  to = from;
}

std::string s;
convert(50, s);
std::cout << s; // outputs "2"
```

The second situation is when a piece of data used throughout a program, such as a unique identifier, is changed type from `int` to `std::string`. Consider the common concept of an "id". While he concept is common and universally understood, there exists no natural internal representation of an identifier. It can be represented by an `int` or by a `std::string`, and sometimes the representation can change in time. If we decide to change the representation in our program, the expextation is that after the change whenever a raw `int` is converted to an id — either in initialization or in the assignment — a compiler should detect a type mismatch and report a compie-time error. But because of the surprising "conversion" this is not the case.

Valid conversions from `int`

There are usages of the assignment from type `int` to `std::string` that are nonetheless valid and behave exactly as intended. These are the cases when we already treat the value stored in an `int` as a character, but we store it in a variable of type `int` either for convenience or because of the peculiar rules of type promotions in C++. The first case is when we use literal `0` to indicate a null character `'\0'`:

```
if (cond1) {
  str = 'A';
}
else if (cond2) {
  str = 'B';
}
else {
  str = 0;  // I mean '\0'
}
```

or:

```
  str = NULL;
```

which — although suspicious — is reported to be used, and is the reason why compilers do not define macro `NULL` as `nullptr`.

Sometimes we may not even be aware that we are producing a value of type `int`:

```
void assign_digit(int d, std::string& s)
// precondition: 0 <= d && d <= 9
{
  constexpr char zero = '0';
  s = (char)d + zero;
}
```

In the example above we might believe that because we are adding two `char`s, the resulting type will also be of type `char`, but the result of the addition of two `char`s is in fact of type `int`. This incorrect expectation is enforced by the way narrowing is defined in C++:

```
// test if char + char == char :
constexpr char zero = '0';
const int d = 9;
char ch {(char)d + zero}; // brace-init prevents narrowing
```

Brace initialization prevents narrowing. The above "test" compiles fine, so no narrowing occur. From this, a programmer could draw an incorrect conclusion that the type of expression `(char)d + zero` must be `char`; but it is not.

Our options

There is a number of ways we can respond to this problem.

Do nothing

That is, do not modify the interface of `std::basic_string`. The potential bugs resulting from the suspicious conversion can be detected by static analyzers rather than compilers. For instance, clang-tidy has checker `bugprone-string-integer-assignment` that reports all places where the suspicious assignment from an `int` is performed. This avoids any correct code breakage, and leaves the option for the bugs to be detected by other tools.

Remove the assignment operator from `charT`

We can just remove the assignment from `charT` altogether. This assignment is suspicious even if no conversions are applied. It is like an assignment of a container element to a container. This warrants the usage of syntax that expresses the element-container relation, like:

```
str.assign(1, ch);
str = {ch};
```

A migration procedure can be provided for changing the program that previously used the suspicious assignment.

Deprecate the assignment

A softer variant of the above would be to declare the assignment from `charT` as deprecated. This does not break any correct code, and allows potential bugs to be detected by the compiler.

Poison te conversion from scalar types to `charT` in the assignment

Do what [RU013] proposes: replace the current signature of the assignment with something equivalent to:

```
template <class T>
  requires is_same_v<T, charT>
constexpr basic_string& operator=(charT c);
```

This may still compromize some valid programs, but the damage is smaller than if the operator was removed altogether. An automated mecanical fix can be easily provided: you just need to apply a cast:

```
str = std::char_traits<char>::to_char_type(i);
```

Poison all conversion but the one from `int`

There is no controversy about disallowing an assignment from `float` or `unsigned int`. Chances that such usages are correct are so small that sacrificing them would be acceptable. The only assignment from non-`charT` that could be potentially correct is the one from `int`, as `int`s are often produced from `char` in unexpected places. Given that, we could poison other assignments, but leave the assignment from `int` intact.

However, all places where this bug has been reported, it was exactly the assignment from `int`, so this option may not be much more attractive than doing nothing.

Offer an alternative interface

If the assignment is narrowed in applicability or removed, this change can be accompanied by adding a dedicated interface for putting a single character into a string. we could add the following signature to `basic_string`:

```
constexpr basic_string& assign_char(charT c);
```

And this avoids any pitfalls, even if an `int` is passed to it:

```
str.assign_char('0' + 0); // we obviously mean a numeric conversion to char
```

It is superior to `str = {ch}` because it allows correct assignments from `int`, and it is superior to `str = {char(ch)}` because it avoids explicit conversion operators.

Acknowledgements

I am grateful to Antony Polukhin and Jorg Brown for their useful feedback.

References

1. [RU013] -- [string.cons].30 (https://github.com/cplusplus/nbballot/issues/13).
2. [CLANG] -- "Extra Clang Tools 10 documentation" (https://clang.llvm.org/extra/clang-tidy/3).